

# 1 作业内容

有一个项目，是平台+业务逻辑方式实现的。其中平台是5526版本，已经在客户处正常运行数年。现在因为新的业务需求需要平台版本的更新。

某天夜间实施版本升级，当平台从5526版本更新到6865版本之后进行测试，发现其中的发短信功能不能正常使用了。升级失败，版本回退到5526。回退之后所有功能恢复正常。

请问这个BUG应该如何排查？请结合《排错》一章所讲的内容，给出一个可行的BUG排查思路。

## 2 BUG 排查思路

我按照课上所述的 DEBUG 的一般思路进行排查。

### 1. 了解清楚 BUG 的表现：

- (a) 出现 BUG 的前置条件：平台版本从 5526 升级到 6865。
- (b) 出现 BUG 时的输入或操作：执行发短信功能。
- (c) 出现 BUG 之后的现象：发短信功能不能正常使用。
- (d) 普遍性和特殊性：
  - i. 旧版本 (5526) 正常，新版本 (6865) 异常，回退后恢复正常。这排除了硬件、网络、操作系统、编译器或标准库的 BUG 导致的可能。
  - ii. BUG 发现时机：夜间实施升级后的测试发现，属于组织内部测试人员或运维人员发现，了解 BUG 表现相对容易。

### 2. 获取这个 BUG 的线索：

- (a) 日志：获取版本 6865 在发短信功能执行前后的日志。检查：
  - i. 平台和业务逻辑在调用发短信接口时是否有错误信息。
  - ii. 怀疑有问题的地方是否缺少日志，或者日志级别是否正确打开。
  - iii. 发短信功能相关的核心代码是否有边界条件的检查以及断言 (`assert`) 的使用。
- (b) Core 文件 (如果发生异常退出)：如果程序异常退出，尝试获取 core 文件，并使用 `gdb` 或 `dbx` 查看崩溃时的函数调用栈状态，定位问题的文件及行号。
- (c) 维护命令：如果有良好的接口设计，使用维护命令查看程序当前状态（例如内存使用、线程状态、发短信模块状态等）。

### 3. 查阅相关版本的代码修改：

- (a) 版本差异对比：利用版本管理工具（如 `git` 或 `SUBVERSION`）对比 5526 版本 和 6865 版本 之间的代码修改。
- (b) 重点关注：
  - i. 与发短信功能接口相关的平台代码修改。
  - ii. 业务逻辑中调用发短信接口的代码是否有变化。
  - iii. 全局配置、库文件等是否有更新或调整。

### 4. 在自己的开发环境复现 BUG：

- (a) 搭建环境：尝试在开发或测试环境部署 6865 版本，并模拟客户环境的配置、数据等。
- (b) 复现操作：执行发短信功能，确认 BUG 能够在开发环境下稳定复现。

### 5. 在复现 BUG 的开发环境下分析：

- (a) 调试工具：使用 `debugger`（如 IDE 内置或 `gdb` / `dbx`）对程序进行单步跟踪，实时查看变量、调用堆栈等。
- (b) Printf 调试：在怀疑的代码位置增加 `printf` 或日志输出语句，通过逻辑推理来缩小问题范围，特别是对于难以复现或工具用途有限的 BUG。
- (c) 如果增加了日志或断言后 BUG 消失，说明当前环境不再是能复现 BUG 的环境，需要恢复到以前并重新修改。

#### 6. 二分法缩小 BUG 的范围：

- (a) 如果代码改动较大或日志线索不明显，可以使用二分法来隔离出导致 BUG 的具体代码变更或模块。
- (b) 例如，从 6865 版本的代码中，逐步移除或注释掉与发短信功能相关的部分代码或新功能，直到 BUG 消失。
- (c) 如果 6865 是从 5526 分支出来的一系列提交，可以尝试使用 `git bisect` 等工具，在 5526 和 6865 之间进行二分查找，定位是哪一个提交引入了 BUG。

#### 7. 解决和预防：

- (a) 解决 BUG：一旦定位到问题代码，进行修复。
  - i. 彻底解决 BUG，避免其再次出现或留下隐患。
  - ii. BUG 越晚解决，所需成本越高。
- (b) 再次测试：修复后进行完善的单元测试用例和自动测试脚本的测试，确保该 BUG 不会在未来的版本中被重新引入。
- (c) 回归测试：在 6865 版本上重新测试发短信功能和其他核心业务功能。
- (d) 最终发布：确认所有测试通过后，安排在客户环境进行二次升级。

## 3 心得

端正态度，努力从自己的代码找问题！